https://github.com/DarkfireDK/Blockchain-Group-project

CDS528 · Blockchain DeFi Smart Contract Development

Flight Delay Insurance DApp

Abstract

This project implements a test-asset flight-delay insurance application in which liquidity providers (LPs) supply capital, travellers purchase NFT-based policies, and an authorised oracle reports qualifying delays. Smart contracts enforce prices, reserve funds, beneficiary ownership and settlement windows. The application combines six Solidity contracts with a Next.js interface and a Hardhat development workflow. Its contribution is the integration of transferable policy benefits, separate net-asset and liquidity accounting, and explicit operational safeguards within a feasible classroom prototype.

Evidence and scope

Recorded verification includes 20 insurance/security regression tests, 22 frontend logic tests and a successful production build on 11 November 2026. A documented Slither run used 101 detectors and retained 11 findings for review. These results support specific implementation claims; they do not establish commercial safety. Local deployment has been demonstrated. Sepolia deployment, explorer verification and remote CI success remain unproven and are not claimed in this report.

Submission information

School of Data Science, Lingnan University. Prepared: 12 November 2026. Group name, group ID, student names and student IDs: to be completed by the team before submission. The assignment specifies a deadline of 8:00 pm on 18 November. This draft follows the supplied assignment rubric and uses 12-point body text.

Reading guide

Sections 1–2 address R1–R2; Sections 3–5 explain the implementation for R3; Sections 6–7 document R4; Section 8 addresses R5; Sections 9–10 and the repository support R6. The accompanying presentation supports R7. Evidence is limited to the records identified in this report, rather than assumed from interface screenshots.

1. Problem, users and scope

Problem definition

Travellers need understandable benefit rules and a clear record of who receives compensation when a covered flight is delayed. A conventional database can record policies, but it leaves payment execution and record changes under an operator's control. This prototype explores a narrower question: can pre-funded smart contracts make conditional payments and transferable benefit ownership inspectable while keeping the operational workflow simple enough for a small student team? It does not measure industry claim-processing times or claim to replace regulated insurance.

Users and goals

The traveller buys coverage and may transfer the benefit before scheduled departure through the marketplace. The LP deposits mock tokens and receives proportional pool shares, earning residual premiums while bearing claims losses. An authorised reporter supplies the off-chain delay result. An administrator configures future-policy parameters and reporter permissions. The desired outcomes are deterministic settlement, reserve-aware withdrawals, transparent ownership and a reproducible test workflow.

Why blockchain

The relevant benefit is programmable custody: token movements, reserve updates and policy transitions succeed or revert together. ERC-20 approvals provide explicit spending permissions, while ERC-721 identifies the current benefit holder (Vogelsteller & Buterin, 2015; Entriken et al., 2018). Publicly inspectable events allow users to reconcile financial activity without relying solely on a frontend label. A central database would still be simpler where users fully trust the operator.

Constraints and assumptions

The system uses freely mintable MockUSDC, never real USDC. Delay information is manually reported; no live airline API or decentralised oracle network is integrated. The fixed 2% premium is a teaching tariff, not actuarial pricing. Insurance eligibility, passenger identity, cancellation, fraud investigation and legal enforceability are outside implementation scope. The design reduces some custody discretion but retains material administrator and oracle trust.

2. Architecture and asset choices

On-chain and off-chain partition

Trust-critical state resides on-chain: policy terms, NFT ownership, LP shares, reserves, roles, listings and settlement. The browser handles forms, wallet selection, validation, human-readable dates and event history. A reporter observes or simulates an external outcome and signs an oracle transaction. Chain time establishes contractual windows; it does not prove an aircraft actually departed. The frontend therefore distinguishes a scheduled-time phase from a confirmed delay report.

Contract responsibilities

MockUSDC provides a six-decimal ERC-20 test asset. PolicyNFT stores policy terms and beneficiary ownership. Insurance validates purchases, snapshots terms and coordinates claims or refunds. InsurancePool holds premiums and LP capital, issues shares and enforces reserved-liquidity constraints. Oracle restricts delay reports to authorised reporters. PolicyMarket implements pre-departure sales and a buyer-specified maximum price. Keeping accounting separate from the policy coordinator makes each responsibility easier to inspect and test.

Asset rationale

A fungible settlement token makes premiums, coverage and LP value comparable without introducing ETH price exposure into the teaching model. The test token has no scarcity or real monetary value. An ERC-721 policy represents one identifiable benefit right and allows the beneficiary to change through ownership. This is a technical right within the application; it does not establish a legally transferable insurance contract. ERC-721 approval is distinct from approval to spend ERC-20 premiums (Entriken et al., 2018).

Role boundaries

Only Insurance receives insurer permissions for pool/NFT operations. Only the policy-snapshotted Oracle may invoke a claim callback. REPORTER_ROLE controls who can submit through that Oracle. Admin roles can grant or revoke permissions, so this prototype is not fully decentralised. On 11 November, the running local instance's reporter was changed to the user-selected test wallet; that runtime permission change is not a guarantee about a fresh deployment.

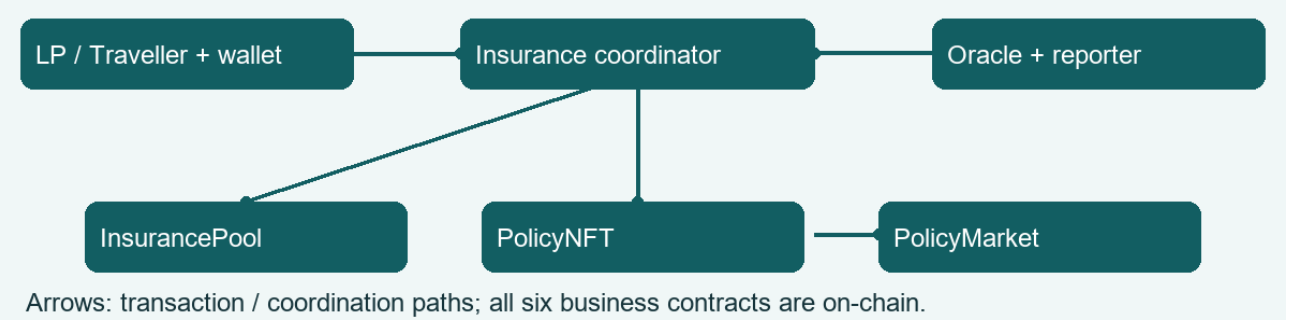


Figure 1. Actors, coordination and value-bearing contracts (project design).

3. Policy lifecycle and transactions

Purchase flow

LP → approve MockUSDC → deposit into InsurancePool → receive shares. Traveller → approve premium spending → call purchasePolicy → validate terms and available capacity → collect premium and reserve coverage/refund → mint policy NFT. All purchase effects occur atomically. Coverage is constrained to 10–5,000 test USDC, with a future coverage start, scheduled departure within the insured period and a maximum seven-day period. Invalid terms or missing approval revert the entire purchase.

Claim and refund flows

A valid delay report is accepted from scheduled departure through coverage end plus 24 hours, including the final second. The delay must reach the policy's snapshotted threshold; the default is 60 minutes. A qualifying report pays the full insured amount to the current NFT owner, closes reserves and burns the NFT. Refund settlement is allowed strictly after that report window and returns the refundable premium to the current holder. The windows do not overlap. Any wallet can trigger expiry, but cannot choose a different beneficiary.

State versus display phase

The raw policy state remains Active until a settlement transaction occurs. The interface derives pending departure, scheduled departure reached, coverage ended and refund available from the latest chain timestamp. Advancing debug time does not itself execute a payout or persist a normal-flight report. Settled states are Claimed or Expired; event-based history preserves results after the NFT is burned. Shared polling checks the latest block every two seconds, without substituting the computer clock.

Secondary transfer

The seller authorises a single NFT and lists a price. A buyer authorises token spending and submits buy(policyId, maxPrice). The market checks active status, current seller ownership, the listing price cap and time strictly before departure. Payment, fee collection and NFT transfer are atomic. These are marketplace restrictions; this report does not claim all direct ERC-721 transfers are blocked after departure. Each policy must currently be reported separately, even when several cover the same flight.

4. Pool accounting and economic model

Pricing and reserves

For coverage C, premium P = ceil(C / 50) in smallest token units, and refundable premium F = floor(P / 2). Risk fee R = P − F. A 500-USDC policy therefore costs 10, of which 5 is refundable. Pool net assets A equal token balance B less refundable reserves Ftotal and pending risk fees Rtotal. Available liquidity L equals B less locked coverage Ctotal and refundable reserves Ftotal, floored at zero. Net assets determine share value; available liquidity determines withdrawal and underwriting capacity.

Share calculations

For deposit D and existing total shares S, new shares = floor[D × (S + 1) / (A + 1)]. Redeeming s shares gives floor[s × (A + 1) / (S + 1)] tokens, subject to available liquidity. The added one is one raw accounting unit, not one displayed USDC; it provides a virtual asset/share offset. Shares and USDC are displayed using six decimals. A share count is a proportional entitlement, not a fixed one-dollar balance.

Observed local example

Local event inspection recorded an initial 2,000-USDC deposit receiving 2,000 shares. Two no-claim settlements added 10 in retained risk fees; two claims each retained a 10 premium while paying 500, producing a combined 980 loss. Net assets became 1,030. A subsequent 1,000 deposit received 1,941.747571 shares, so the first account held 3,941.747571 shares worth approximately 2,030. Another 1,000 deposit received the same additional shares. Total net assets were 3,030 and total shares 5,883.495142. These are local historical observations, not expected returns.

Revenue and loss

LP revenue comes from settled policy premiums, not later LP deposits. An unclaimed 500 policy adds 5 to LP assets after refund. A claimed policy reduces assets by 490 after recognising its 10 premium. Thus more shares after a loss do not mean a profit. Under this simplified tariff, expected pool change per policy is 5 − 495q, where q is the probability of a qualifying claim. Ignoring all other costs, break-even is q = 5/495 ≈ 1.01%; no empirical q was estimated. Fully reserving coverage constrains exposure but does not make the tariff profitable.

5. Implementation and user experience

Engineering choices

The implementation uses Solidity, OpenZeppelin building blocks and Hardhat, with a TypeScript/React/Next.js frontend. OpenZeppelin role-based access control separates administration, underwriting and reporting (OpenZeppelin, n.d.). SafeERC20 handles supported token transfers; reentrancy guards and state updates before external transfers reduce callback risk. Policy terms snapshot Oracle address and minimum delay at purchase so future parameter changes do not silently alter existing policies.

Events and atomicity

Deposit, withdrawal, policy funding, claim, refund, delay-report and marketplace events provide an audit trail. Oracle reporting is single-use; unsuccessful below-threshold or out-of-window reports revert without consuming the report opportunity. Settled NFTs are burned, preventing reuse. Pool deposits verify exact token receipt and reject zero-share issuance; fee-on-transfer and rebasing assets are intentionally unsupported. These restrictions fit the single MockUSDC deployment and should not be generalised to arbitrary tokens.

Workflow improvements

Insufficient underwriting capacity now offers a direct route to the liquidity deposit panel. Forms preserve context and explain the separate approval and transaction steps. The operational panel queries a policy before presenting its beneficiary, individual threshold and report cutoff. It disables invalid reports, distinguishes normal-flight guidance from an actual on-chain record, and offers a permissionless expiry action when eligible. Wallet writes check the selected account, connector authorisation and chain before requesting a signature.

Gas awareness and limitations

Each purchase or settlement updates a bounded number of records, rather than iterating over all policyholders. Integer arithmetic avoids floating-point asset amounts. Event history is read off-chain. These choices improve predictability, but the project has not produced a comparative gas benchmark and cannot claim a measured gas reduction. Per-policy reporting requires multiple transactions for multiple policies. A future batch design must be bounded, preserve per-policy checks and specify how partial failures are handled.

6. Validation and reproducibility

Observed results

The final recorded insurance regression run on 11 November completed 20 tests successfully on an isolated Hardhat network. A separate run passed 22 frontend logic tests: eight flight-management validations, eight policy-phase boundaries and six market-eligibility checks. Type checking and the production build succeeded; the build generated seven static pages. These are different test layers, not 42 end-to-end browser tests. The report reuses saved logs and does not claim a fresh rerun on the report date.

Meaningful coverage

Contract tests verify premium approval, fixed tariff and date constraints, NFT ownership, reserve accounting, claim/refund amounts and prevention of repeated settlement. They check unauthorised callbacks, reporter permissions, policy term snapshots, paused purchases with preserved exits, stale or repriced listings, NFT callback reentrancy and zero-share donation dilution. A deterministic 20-policy mixed-settlement test checks accounting behaviour across multiple operations; it is not an Echidna campaign or exhaustive fuzzing.

Boundary and integration testing

Tests include the instant of departure, the last permissible report second and the first permissible refund second. Below-threshold reports revert and can later be followed by a qualifying report. Frontend logic tests reject invalid integer delays, missing chain time, incorrect Oracle bindings and settled policies. Earlier project records document local browser purchase and listing flows. The newly added flight-management screen has not yet received full MetaMask/browser end-to-end acceptance, and this gap remains explicit.

Reproduction and evidence

Use Node 22.14.0 or a supported newer version and the repository's Yarn 4.13.0. From the root, install with yarn install --immutable, compile with yarn compile, then run yarn test:insurance and yarn next:check-types. The three frontend suites can be run with tsx --test under the Hardhat workspace. Exact commands and acceptance steps are in docs/flight-management-testing.md. Evidence includes flight-management-contract-tests.log, flight-management-build.log, slither.json and security-review.md. Logs support the stated checks; no coverage percentage or latency metric is invented.

7. Security and trust evaluation

Automated scan

The documented Slither 0.11.5 run analysed six business contracts using 101 detectors. It retained 11 findings: two High, four Medium, four Low and one Informational. Slither is a static analysis framework, not proof of correctness (Crytic, n.d.). Findings and dispositions are preserved rather than suppressed. Reviewed source hashes bind the review record to the scanned code; changes require renewed review. No independent security audit is claimed.

Threat 1: unauthorised or false reporting

AccessControl and policy-snapshotted Oracle checks reject callers without the required authority. Window and threshold checks constrain otherwise authorised reports. However, an authorised reporter can still lie about an external event, and an administrator can grant permissions to a malicious reporter. Real-world data integrity remains unresolved. Production work would require authenticated aviation data, independent agreement or a dispute mechanism; such controls are proposals, not integrated features.

Threat 2: reentrancy and token movement

Reentrancy guards, atomic state transitions and SafeERC20 reduce repeated withdrawals and callback abuse. A malicious NFT-receiver regression tests purchase reentry. Slither's arbitrary-send findings were reviewed against role restrictions and the payer/seller/buyer paths; one token finding arose from an ERC-721 transferFrom name match. Exact-receipt and zero-result equality checks intentionally reject unsupported tokens and destructive rounding. Reviewed findings remain visible and depend on correct role administration.

Threat 3: economic and transaction-order abuse

Separating net assets from locked coverage prevents a reservation change from artificially repricing LP shares. Pending premiums are excluded until settlement, while virtual units and zero-share checks address cheap dilution. Buyer maxPrice and live ownership/time checks reject stale or changed listings. These measures do not solve correlated flight losses, adverse selection, multiple identities buying related policies, or fixed-tariff underpricing. Timestamp comparisons are necessary for the windows; block timestamps are not exact aviation evidence.

```
Recorded Slither output excerpt (docs/evidence/slither.log)

Scanned 6 source files with 101 detectors; 11 findings. Output: C:\Users\Al\Documents\ChatGPT\CDS
528 Blockchain\insurance-dapp\docs\evidence\slither.json
Unreviewed High/Medium: 0; findings are retained even when reviewed.
```

Figure 2. Excerpt of the recorded automated scan output; findings remain retained.

8. Deployment, operations and provenance

Local deployment

The project deploys MockUSDC, PolicyNFT, InsurancePool, Insurance, Oracle and PolicyMarket, then configures their roles. The one-click launcher starts the local workflow; the manual sequence is yarn chain, yarn deploy and yarn start. Hardhat provides the development and testing framework (Nomic Foundation, n.d.). Local debug time controls verify a standalone, non-forked chain with ID 31337 and move time forward only. They never operate on Sepolia. Resetting a development chain can erase state and undo manually changed permissions.

Testnet gap

Sepolia is configured, but no Sepolia deployment transaction or verified contract address is evidenced in the current project records. Local addresses and transactions must not be submitted as testnet evidence. To complete R5, a team member must configure a dedicated test wallet and RPC, deploy the mock-asset contracts, verify their source, record explorer URLs in README and test the frontend against that deployment. Private keys, seed phrases, decrypted wallet files and environment secrets must remain outside the public repository.

Operations rationale

Contracts are immutable for implementation simplicity; there is no upgrade proxy. An administrator may pause new purchases while preserving existing claims and refunds. A defect response would stop new business, communicate limitations and deploy a replacement subject to user-directed migration. There is no promise that an existing policy can be rewritten. The GitHub repository was uploaded previously, but current local changes may not be pushed. Remote CI is configured without a verified successful run.

Provenance and submission readiness

The starting implementation came from GtiHuberZZ/insurance-dapp, with baseline revision 4f85fc790c8e7bcd0e50f7add785fb6fc9585b26. The team’s work extends and corrects that baseline rather than claiming every component as original. The supplied PDF requires a public repository, while project records describe the personal repository as private: visibility and instructor access must be resolved before submission. OpenZeppelin and Next.js use MIT-licensed distributions; retain their notices. An explicit top-level project licence and upstream redistribution review remain to be confirmed.

9. Evaluation, conclusion and team review

What the prototype demonstrates

The project demonstrates conditional mock-token custody, transferable policy benefits and reserve-aware LP accounting within a coherent interface. The observed loss-and-deposit example is a useful validation: a larger share count can coexist with a lower portfolio value, and UI labels must explain that relationship. The distinction between time-derived phases and transaction-driven final states likewise avoids implying that blockchain time is a trusted flight-status feed.

What remains incomplete

The largest remaining gaps are verified testnet deployment, a public and up-to-date repository, remote CI evidence and end-to-end acceptance of the new flight-management page. The system also lacks live oracle integration, empirical pricing calibration, gas benchmarking and independent auditing. Browser-only validation is insufficient for trust-critical conditions, so contracts repeat the essential permission, amount and time checks. These limitations prevent a commercial-readiness claim even though the recorded regression suites pass.

Next development steps

First complete the assignment-facing evidence: push reviewed changes without secrets, make the repository publicly accessible, run CI and record real Sepolia verification links. Then validate the full two-wallet browser workflow, including denial of signatures and network/account changes. Further research should examine correlated claims, refundable-premium incentives and a stronger oracle design. Any new pricing mechanism should be evaluated against explicitly stated claim probabilities and loss limits rather than selected to make a demo appear profitable.

Conclusion and contribution record

Smart contracts make the implemented custody and settlement rules inspectable, but cannot establish the truth of an off-chain flight result. Correct accounting and explicit trust assumptions are therefore as important as the transaction interface. Before submission, each of the 3–5 group members must add their actual name, student ID and verified contribution: scope/integration, contract design, frontend, testing/security, or deployment/documentation. Roles may be shared. No names, percentages or Git contributions are fabricated here. Every member should present a portion of the accompanying 12-minute deck.

10. References and AI disclosure

References

Crytic. (n.d.). Slither [Computer software]. GitHub. https://github.com/crytic/slither

Entriken, W., Shirley, D., Evans, J., & Sachs, N. (2018). ERC-721: Non-fungible token standard. Ethereum Improvement Proposals. https://eips.ethereum.org/EIPS/eip-721

Nomic Foundation. (n.d.). Getting started with Hardhat 3. https://hardhat.org/docs/getting-started

OpenZeppelin. (n.d.). Access control. OpenZeppelin Contracts documentation. https://docs.openzeppelin.com/contracts/5.x/access-control

Vogelsteller, F., & Buterin, V. (2015). ERC-20: Token standard. Ethereum Improvement Proposals. https://eips.ethereum.org/EIPS/eip-20

Project and assignment sources

GtiHuberZZ. (n.d.). insurance-dapp [Source code]. GitHub. https://github.com/GtiHuberZZ/insurance-dapp

DarkfireDK. (2026). Blockchain-Group-project [Source code and local engineering records]. GitHub. https://github.com/DarkfireDK/Blockchain-Group-project

School of Data Science, Lingnan University. (n.d.). CDS528 group assignment: Blockchain DeFi smart contract development [Course handout]. Supplied CDS_528_Group_Project.pdf.

Technical evidence: docs/test-results.md, docs/security-review.md, docs/flight-management-testing.md and associated logs. These are internal project records, not independent research studies.

AI-assisted work disclosure

OpenAI ChatGPT/Codex assisted with implementation and debugging, test preparation, analysis, and drafting this report and presentation. Automated checks and recorded outputs support the specific claims identified above. The team remains responsible for reviewing code, calculations, source attribution, licence compatibility and final wording. This draft does not claim that every generated statement has received independent human review. Record each member's actual review and contribution before submission. No private keys or real-asset transactions are included.